Mid BigRC Part1

Yunxiang Yao

1. Linux (Chapter 02)

Exercise 2.1: Directory Navigation

Consider the following directory structure:

```
/home/
  alice/
    project/
      src/
        main.cpp
      include/
        header.h
    notes.txt
  bob/
    data.txt
```

Alice's current working directory is `/home/alice/project/src`.

Exercise 2.1: Questions

1. Write a command to list all files (including hidden) in Alice's home directory using a **relative path**.
2. Write a command to copy `header.h` to `src/` using a relative path.
3. Write a single command to move `notes.txt` into the `project/` directory.

Exercise 2.2: File Permissions

Given the following `ls -l` output:

```
-rwxr-x--- 1 alice staff 2048 Mar 15 10:30 script.sh
drwxrwx--x 2 alice staff 4096 Mar 15 09:00 shared/
-rw-r--r-- 1 bob   staff 1024 Mar 14 08:00 notes.txt
```

Answer the following questions:

1. Can user `bob` execute `script.sh`? Why or why not?
2. Can user `bob` create a new file inside `shared/`?
3. What does the `x` permission on a **directory** mean?
4. If `carol` (not in `staff` group) runs `cd shared/`, will it succeed?

Exercise 2.3: I/O Redirection

What is the difference between the following commands? Explain what each does.

1. `cat file1.txt file2.txt > combined.txt`
2. `cat file1.txt > file2.txt`
3. `cat < file1.txt > file2.txt`
4. `diff file1.txt file2.txt > result.txt`

Bonus: If `result.txt` is empty after command 4, what does that tell you?

Exercise 2.4: Wildcard Matching

Given these files in the current directory:

```
test.cpp  test1.cpp  test12.cpp  main.cpp
data.txt  test.h  test1.h  utils.cpp
```

Which files are matched by each command?

1. `ls test*.cpp`
2. `ls *.cpp`
3. `ls test?.cpp`
4. `ls test*`

2. Developing Programs (Chapter 03)

Exercise 3.1: Compilation Process

Consider the following files:

math_utils.h

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int square(int x);
int cube(int x);
#endif
```

main.cpp

```
#include <iostream>
#include "math_utils.h"
using namespace std;
int main() {
    cout << square(5);
    cout << cube(3);
    return 0;
}
```

Exercise 3.1: Compilation Process (cont.)

math_utils.cpp

```
#include "math_utils.h"
int square(int x) {
    return x * x;
}
int cube(int x) {
    return x * x * x;
}
```

Questions:

1. Write the g++ command to compile these files into an executable called `calc`.
2. Write commands to compile separately into object files, then link.

Exercise 3.2: Header Guard

What's wrong with the following code? What error will the compiler produce?

point.h

```
struct Point {
    int x;
    int y;
};
```

line.h

```
#include "point.h"
struct Line {
    Point start;
    Point end;
};
```

draw.cpp

```
#include "point.h"
#include "line.h"
int main() {
    Point p;
    Line l;
    return 0;
}
```

How would you fix `point.h`?

Exercise 3.3: Makefile Analysis

Analyze this Makefile and answer the questions:

Makefile

```
all: program

program: main.o utils.o data.o
    g++ -o program main.o utils.o data.o

main.o: main.cpp utils.h data.h
    g++ -c main.cpp

utils.o: utils.cpp utils.h
    g++ -c utils.cpp

data.o: data.cpp data.h
    g++ -c data.cpp

clean:
    rm -f program *.o
```

Exercise 3.3: Questions

1. If you modify `utils.h`, which files will be recompiled when you run `make`?
2. If you modify only `data.cpp`, which commands will be executed?
3. Why is `main.o` dependent on both `utils.h` and `data.h`?
4. What happens if you run `make clean` followed by `make`?

3. C++ Basics (Chapter 04)

Exercise 4.1: lvalue vs rvalue

For each expression, state whether it is an **lvalue** or **rvalue**:

```
int a = 5;
int b = 10;
int arr[3] = {1, 2, 3};
int *ptr = &a;
```

1. `a`
2. `a + b`
3. `arr[0]`
4. `arr[a - 4]`
5. `*ptr`
6. `&a`
7. `ptr`
8. `++a`
9. `a++`

Exercise 4.2: Pass-by-Value vs Pass-by-Reference

What is the output of the following program?

```
#include <iostream>
using namespace std;

void foo(int x, int &y, int *z) {
    x = x + 1;
    y = y + 2;
    *z = *z + 3;
    z = &x;  // What effect does this have?
}

int main() {
    int a = 10, b = 20, c = 30;
    foo(a, b, &c);
    cout << a << " " << b << " " << c << endl;
    return 0;
}
```

Exercise 4.3: Pointers and Arrays

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;
int *q = &arr[2];
```

Evaluate each expression:

1. `*p`
2. `*(p + 2)`
3. `p[3]`
4. `*q`
5. `q - p`
6. `*(q + 1)`
7. `q[-1]`

Exercise 4.4: References

Identify which statements are **valid** or **invalid**. Explain why.

```
int a = 5;
int b = 10;
int &ref1 = a;          // (1)
int &ref2;              // (2)
int &ref3 = 10;         // (3)
int &ref4 = a + b;      // (4)
ref1 = b;               // (5)
int &ref5 = ref1;       // (6)
```

After all valid statements execute, what are the values of `a`, `b`, `ref1`, and `ref5`?

Exercise 4.5: Structs and Pointers

```
struct Student {
    char name[20];
    int score;
};

void updateScore(Student *s, int bonus) {
    s->score += bonus;
}

int main() {
    Student alice = {"Alice", 85};
    Student *ptr = &alice;
    updateScore(ptr, 10);
    (*ptr).score += 5;
    ptr->score -= 3;
    // What is alice.score now?
}
```

4. const Qualifier (Chapter 05)

Exercise 5.1: const Basics

Identify which statements are **valid** or **invalid**:

```
const int MAX = 100;        // (1)
const int MIN;              // (2)
MAX = 200;                  // (3)

int x = 50;
const int &ref1 = x;        // (4)
const int &ref2 = 42;       // (5)
const int &ref3 = x + 10;   // (6)
int &ref4 = MAX;            // (7)

ref1 = 60;                  // (8)
x = 60;                     // (9)
```

Exercise 5.2: Pointer to const vs const Pointer

What is the difference between these declarations?

```
int value = 10;

const int *p1 = &value;     // Type A
int const *p2 = &value;     // Type B
int *const p3 = &value;     // Type C
const int *const p4 = &value; // Type D
```

For each type, determine if the following operations are valid:

1. `*px = 20;` (modify the pointed-to value)
2. `px = &other;` (change what pointer points to)

Exercise 5.3: const in Function Parameters

Which function calls are **valid**? Which are **invalid**?

```
void func1(int *p) { *p = 100; }
void func2(const int *p) { cout << *p; }
void func3(int &r) { r = 100; }
void func4(const int &r) { cout << r; }

int main() {
    int x = 10;
    const int y = 20;

    func1(&x);  /* (1) */    func1(&y);  /* (2) */
    func2(&x);  /* (3) */    func2(&y);  /* (4) */
    func3(x);   /* (5) */    func3(y);   /* (6) */
    func4(x);   /* (7) */    func4(y);   /* (8) */
    func4(30);  /* (9) */    func3(30);  /* (10) */
}
```

Exercise 5.4: typedef with const

Given:

```
typedef int * intptr_t;
typedef const int * ptr_constint_t;
```

What types do the following declarations create?

```
const intptr_t p1;       // (1)
intptr_t const p2;       // (2)
ptr_constint_t p3;       // (3)
const ptr_constint_t p4; // (4)
```

Which are equivalent to `const int *`? Which to `int * const`?

Exercise 5.5: Comprehensive const Analysis

```
struct Data {
    int value;
};

void process(const Data &d) {
    // Which lines are valid inside this function?
    cout << d.value;          // (1)
    d.value = 10;             // (2)
    Data copy = d;            // (3)
    copy.value = 20;          // (4)
    const Data *ptr = &d;     // (5)
    Data *ptr2 = &d;          // (6)
}
```

Thank you!